

ESP32 Controller <-> Nxtion 2.8" HMI Touch Dashboard & LED-Dimmer (Bi-Direktional)



Ein robustes, bibliotheksfreies Arduino C++ Projekt zur **bi-direktionalen seriellen Kommunikation** zwischen einem **ESP32 WROOM** und einem **Nxtion 2.8" TFT HMI-Touch-Display** über den Hardware-UART2 mit **PWM-Helligkeitssteuerung** einer externen LED an **GPIO 32**.

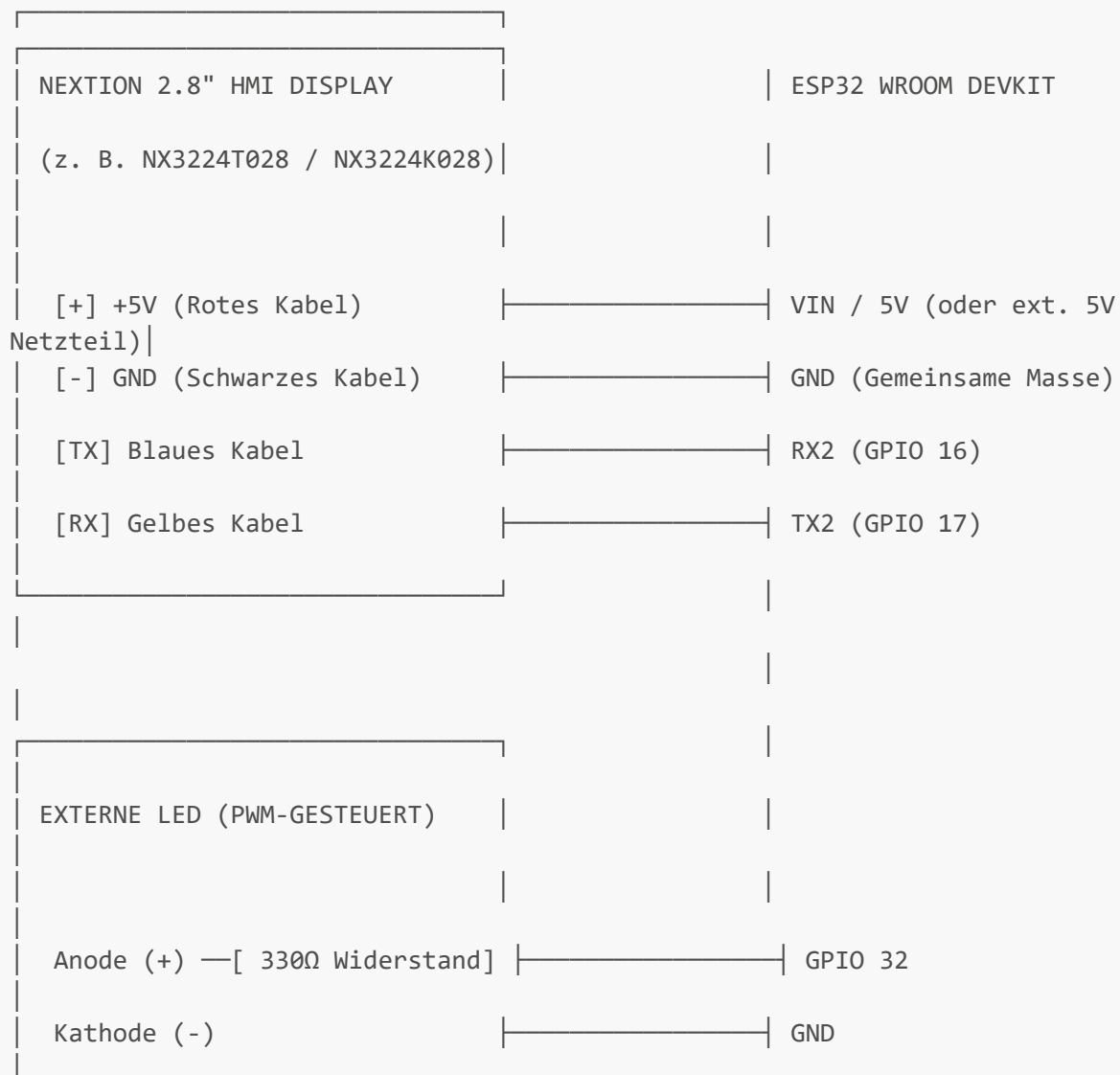


Features

- **Echter Hardware-UART2 (HardwareSerial 2):** Verwendet GPIO 16 (RX2) und GPIO 17 (TX2). Der primäre USB-Port (UART0 / Serial) bleibt zu 100 % frei für Delphi, den Seriellen Monitor oder PC-Schnittstellen.
- **PWM LED-Helligkeitssteuerung (Dimm-Funktion):**

- Externe LED an **GPIO 32** mit Vorwiderstand.
- Helligkeit (0–100 %) stufenlos regelbar über den Nextion **Slider h0**.
- Ein-/Ausschalten über den Nextion **Touch-Button bt0**.
- **Bi-direktionaler Datenaustausch:**
 - **ESP32 → Nextion:** Zyklisches Aktualisieren von Sensorwerten (Temperatur **t0.txt**, Drehzahl **n0.val**, Fortschrittsbalken **j0.val**).
 - **Nextion → ESP32:** Verarbeiten von Touch-Events (**0x65** Pakete) und Slider-Werten (**0x71** Pakete).
 - **ESP32 → Nextion Rückmeldung:** Status- und Prozentanzeige auf dem Display (**t1.txt="LED: AN (75%)"**).
- **Universeller & robuster Puffer-Parser:** Erkennt zuverlässig die 3x **0xFF** Endekennung von Nextion ohne blockierende Delays.
- **Keine externen Libraries nötig:** Verwendet reinen nativen ESP32-UART- und PWM-Code ohne instabile Drittanbieter-Bibliotheken.

🔧 Hardware-Verkabelung (Pinout)



Komponente / Pin	Kabelfarbe	ESP32 WROOM Pin	Funktion
Nextion +5V	Rot	VIN (5V)	Spannungsversorgung Display (mind. 500 mA)
Nextion GND	Schwarz	GND	Gemeinsame Masse
Nextion TX	Blau	GPIO 16 (RX2)	Nextion sendet Touch- & Slider-Daten an ESP32
Nextion RX	Gelb	GPIO 17 (TX2)	ESP32 sendet Messwerte & Befehle an Nextion
LED Anode (+)	mit 330Ω Vorwiderstand	GPIO 32	PWM-Ausgang zur Helligkeitsregelung (0–255)
LED Kathode (-)	direkt	GND	Masse der LED

Einrichtung im Nextion Editor

Erstelle im **Nextion Editor** dein Dashboard auf **page 0** mit folgenden Komponenten:

Komponenten-Typ	Objektname (objname)	Beschreibung	Wichtige Attribute / Events
Text	t0	Temperaturanzeige (z. B. "23.4 °C")	txt_max1: 15
Number	n0	Numerischer Wert (Drehzahl / RPM)	vvs1: 0
Progressbar	j0	Fortschrittsbalken (0–100 %)	val: 0, maxval: 100
Dual-State Button	bt0	LED Ein/Aus Taster	Touch Press Event: ☒ Send Component ID
Slider	h0	Helligkeits-Schieberegler (0–100)	Touch Press / Release: ☒ Send Component ID
Text	t1	Statusanzeige ("LED: AN (50%)")	txt_max1: 20

 **Slider-Einstellung:** Klicke im Nextion Editor auf den Slider **h0** und aktiviere im Eigenschaftfenster bei **Touch Press Event** oder **Touch Release Event** das Häkchen ☒ **Send**

Component ID. Der ESP32 fragt bei jeder Berührung automatisch den genauen Positionswert mit `get h0.val` ab!

Das Nextion-Protokoll im Detail

1. ESP32 an Nextion (Befehle senden)

Jeder Befehl an das Display ist ein lesbarer ASCII-String, der **zwingend mit 3 Bytes des Wertes 0xFF (255)** abgeschlossen werden muss:

- **Text ändern:** `t0.txt="23.4 °C" + 0xFF 0xFF 0xFF`
- **Zahlenwert ändern:** `n0.val=1450 + 0xFF 0xFF 0xFF`
- **Slider-Wert abfragen:** `get h0.val + 0xFF 0xFF 0xFF`

2. Nextion an ESP32 (Touch & Rückmeldungen)

- **Touch-Event (7 Bytes):** `0x65 [PageID] [ComponentID] [EventType] 0xFF 0xFF 0xFF`
- **Numerische Antwort (8 Bytes):** `0x71 [B0] [B1] [B2] [B3] 0xFF 0xFF 0xFF` (4 Bytes Integer Little-Endian als Antwort auf `get h0.val`)

Vollständiger ESP32 C++ Code (`ESP32_Nextion_UART2.ino`)

```
/* =====
 * ESP32 WROOM <-> NEXTION 2.8" DISPLAY (BI-DIREKTIONALE STEUERUNG)
 * Hardware-UART2: RX2 = GPIO 16, TX2 = GPIO 17 (9600 Baud)
 * Externe LED an GPIO 32 mit PWM-Helligkeitssteuerung (Slider h0)
 * USB-UART0: Debugging & Delphi Verbindung (115200 Baud)
 * =====
 */

#include <Arduino.h>

// Pins fuer Nextion Display an HardwareSerial 2 des ESP32
#define NEXTION_RX_PIN 16 // Verbinden mit Nextion TX (Blau)
#define NEXTION_TX_PIN 17 // Verbinden mit Nextion RX (Gelb)
#define NEXTION_BAUD 9600

// Externe LED an GPIO 32 mit Vorwiderstand (z. B. 330 Ohm)
#define LED_PIN 32

HardwareSerial NextionSerial(2); // Nutzt Hardware-UART2 des ESP32

// Globale Zustaende
static bool led_state = true; // LED Ein / Aus
static int led_brightness = 50; // 0 bis 100 % (vom Slider h0)

// -----
-
```

```

// 1. PWM-STEUERUNG FUER DIE LED AN GPIO 32
// -----
-

void update_led_pwm() {
    if (led_state) {
        // Helligkeit 0..100% auf 8-Bit PWM (0..255) umrechnen
        int pwm_val = map(led_brightness, 0, 100, 0, 255);
        analogWrite(LED_PIN, pwm_val);
        Serial.printf("--> [LED GPIO 32] STATUS: AN | Helligkeit: %d%% (PWM:
%d/255)\r\n",
                        led_brightness, pwm_val);
    } else {
        analogWrite(LED_PIN, 0); // Ausgeschaltet (0 PWM)
        Serial.printf("--> [LED GPIO 32] STATUS: AUS (0 PWM)\r\n");
    }
}

// -----
-

// 2. HILFSFUNKTIONEN ZUM SENDEN AN DAS NEXTION DISPLAY
// -----
-

// Sendet Befehl + 3x 0xFF Abschluss
void nextion_send_cmd(const String& cmd) {
    NextionSerial.print(cmd);
    NextionSerial.write(0xFF);
    NextionSerial.write(0xFF);
    NextionSerial.write(0xFF);
}

// Textfeld aktualisieren (z.B. t0.txt="23.4 C")
void nextion_set_text(const String& component, const String& text) {
    String cmd = component + ".txt=\"" + text + "\"";
    nextion_send_cmd(cmd);
}

// Zahlenwert aktualisieren (z.B. n0.val=1450, h0.val=50, bt0.val=1)
void nextion_set_number(const String& component, int32_t val) {
    String cmd = component + ".val=" + String(val);
    nextion_send_cmd(cmd);
}

// -----
-

// 3. EMPFANGS-LOGIK: BUTTONS & SLIDER VOM NEXTION DISPLAY VERARBEITEN
// -----
-

void handle_nextion_data() {
    static uint8_t rx_buffer[32];
    static uint8_t rx_index = 0;

```

```

static uint8_t ff_count = 0;

while (NextionSerial.available()) {
    uint8_t b = NextionSerial.read();

    // Auf 3x 0xFF Endekennung pruefen
    if (b == 0xFF) {
        ff_count++;
        if (ff_count == 3) {
            // FALL 1: Touch-Event Paket (0x65 [Page] [CompID] [Event])
            if (rx_index >= 4 && rx_buffer[0] == 0x65) {
                uint8_t pageId    = rx_buffer[1];
                uint8_t compId    = rx_buffer[2];
                uint8_t eventType = rx_buffer[3]; // 1 = Press, 0 = Release

                Serial.printf("[NEXTION TOUCH] Page: %d | Component-ID: %d
| Event: %s\r\n",
                            pageId, compId, (eventType == 1 ? "PRESS" :
"RELEASE"));

                // Wenn Slider h0 beruehrt wurde -> aktuellen Wert mit "get
h0.val" anfordern
                nextion_send_cmd("get h0.val");

                // Button bt0 schaltet die LED EIN / AUS
                if (eventType == 1) { // Bei Tastendruck
                    led_state = !led_state;
                    update_led_pwm();

                    // Display-Status aktualisieren
                    if (led_state) {
                        nextion_set_text("t1", "LED: AN (" +
String(led_brightness) + "%)");
                        nextion_set_number("bt0", 1);
                    } else {
                        nextion_set_text("t1", "LED: AUS");
                        nextion_set_number("bt0", 0);
                    }
                }
            }
            // FALL 2: Numerische Rueckmeldung vom Display (0x71 [4 Bytes
Integer])

            // Wird von Nextion als Antwort auf "get h0.val" gesendet!
            else if (rx_index >= 5 && rx_buffer[0] == 0x71) {
                uint32_t val = (uint32_t)rx_buffer[1] |
((uint32_t)rx_buffer[2] << 8) |
((uint32_t)rx_buffer[3] << 16) |
((uint32_t)rx_buffer[4] << 24);

                led_brightness = constrain((int)val, 0, 100);
                Serial.printf("[NEXTION SLIDER h0] Neuer Helligkeitswert
empfangen: %d%%\r\n", led_brightness);

```

```

        // PWM Helligkeit auf GPIO 32 anwenden
        update_led_pwm();

        // Wenn LED an ist, Textanzeige mit Prozent aktualisieren
        if (led_state) {
            nextion_set_text("t1", "LED: AN (" +
String(led_brightness) + "%)");
        }
    }
    // FALL 3: Direkter ASCII-String (z.B. "H:75" aus Touch Move
Event)
    else if (rx_index >= 3 && rx_buffer[0] == 'H' && rx_buffer[1]
== ':') {
        rx_buffer[rx_index] = '\0';
        int val = atoi((char*)&rx_buffer[2]);
        led_brightness = constrain(val, 0, 100);
        update_led_pwm();
        if (led_state) {
            nextion_set_text("t1", "LED: AN (" +
String(led_brightness) + "%)");
        }
    }

    // Puffer leeren
    rx_index = 0;
    ff_count = 0;
    continue;
}
} else {
    ff_count = 0;
    if (rx_index < sizeof(rx_buffer) - 1) {
        rx_buffer[rx_index++] = b;
    }
}
}

// -----
// 4. SETUP & HAUPTSCHLEIFE
// -----

void setup() {
    // 1. Debug-Monitor via USB (UART0)
    Serial.begin(115200);

    // 2. LED Pin an GPIO 32 initialisieren
    pinMode(LED_PIN, OUTPUT);
    update_led_pwm(); // Startet mit 50% Helligkeit

    // 3. Hardware-UART2 fuer Nextion initialisieren
    NextionSerial.begin(NEXTION_BAUD, SERIAL_8N1, NEXTION_RX_PIN,

```

```

NEXTION_TX_PIN);

    delay(500); // Warten auf Display-Start

    Serial.println("=====");
    Serial.println(" ESP32 WROOM <-> Nextion 2.8\" HMI Gestartet ");
    Serial.println(" LED an GPIO 32 mit Slider h0 Dimm-Funktion ");
    Serial.println("=====");

    // Initiale Werte auf dem Nextion Display setzen
    nextion_set_text("t0", "ESP32 Start");
    nextion_set_text("t1", "LED: AN (50%)");
    nextion_set_number("bt0", 1);
    nextion_set_number("h0", 50); // Slider h0 auf 50%
    nextion_set_number("n0", 0);
    nextion_set_number("j0", 0);
}

static uint32_t last_sensor_time = 0;
static float sim_temperature = 23.4;
static int sim_rpm = 1450;
static int sim_progress = 65;

void loop() {
    // 1. Laufend Touch-Ereignisse und Slider-Werte vom Nextion abfragen
    handle_nextion_data();

    // 2. Alle 1000 ms neue Sensorwerte an Nextion senden
    if (millis() - last_sensor_time >= 1000) {
        last_sensor_time = millis();

        // Messwerte simulieren
        sim_temperature += ((random(0, 20) - 10) / 10.0);
        if (sim_temperature < 18.0) sim_temperature = 18.0;
        if (sim_temperature > 35.0) sim_temperature = 35.0;

        sim_rpm += (random(-50, 50));
        sim_progress = (sim_progress + 5) % 105;

        // An Nextion senden
        char tempStr[16];
        snprintf(tempStr, sizeof(tempStr), "%.1f C", sim_temperature);
        nextion_set_text("t0", tempStr);
        nextion_set_number("n0", sim_rpm);
        nextion_set_number("j0", sim_progress);

        Serial.printf("[SENSOR] Temp: %s | RPM: %d | Progress: %d%% | LED: %s
(%d%%)\r\n",
                        tempStr, sim_rpm, sim_progress,
                        led_state ? "AN" : "AUS", led_brightness);
    }
}

```


Fehlerbehebung & Häufige Fallstricke

Symptom	Ursache	Lösung
Slider ändert die Helligkeit nicht	Send Component ID nicht aktiv	Im Nextion Editor Slider h0 anklicken → Touch Event → ☒ Send Component ID anhängen.
LED an GPIO 32 leuchtet nicht	Polarität der LED / Vorwiderstand	Längeres Beinchen (Anode) über 330Ω an GPIO 32 , kürzeres Beinchen (Kathode) an GND .
Touch wird im Monitor erkannt, aber Status ändert sich nicht	Falsche Component-ID	Im Seriellen Monitor nach der Zahl bei Component-ID: X schauen und Namen (t1 , bt0 , h0) im Nextion Editor prüfen.
Display zeigt nach ESP32-Start gar nichts an	TX/RX vertauscht	GPIO 16 (ESP32 RX2) muss an Nextion TX (Blau), GPIO 17 (ESP32 TX2) an Nextion RX (Gelb).

Lizenz

MIT License. Freie Verwendung für private und industrielle Projekte.